

Draw It or Lose It

CS 230 Project Software Design Template

Scott Masciulli

10/12/2026

Version 1.0

Table of Contents

CS 230 Project Software Design Template 1

Table of Contents 2
Document Revision History 2
Executive Summary 3
Requirements 3
Design Constraints 3
System Architecture View 3
Domain Model 3
Evaluation 4
Recommendations 5

Document Revision History

Version	Date	Author	Comments
1.0	10/12/2026	Scott Masciulli	Completed the software design template for The Gaming Room's Draw It or Lose It application.

Instructions

Fill in all bracketed information on page one (the cover page), in the Document Revision History table, and below each header. Under each header, remove the bracketed prompt and write your own paragraph response covering the indicated information.

Executive Summary

Draw It or Lose It is required to function as a web-based application within a distributed environment, capable of scaling to accommodate thousands of players. The application must be compatible with Windows, macOS, Linux, Android, and iOS platforms via modern responsive web browsers. The server infrastructure should ensure reliable hosting, networking, security, storage, and scalability. The development process must address variations in browsers, screen sizes, input methods, network conditions, and mobile device performance. Additionally, the development team requires tools and programming languages that facilitate server management, responsive web interface design, database or persistent storage integration, testing, source control, and deployment.

Requirements

I identified the main business and technical requirements as cross-platform access, support for multiple players and game sessions, consistent management of shared game information, reliable communication between clients and the server, and a design that can be maintained as the application grows. I also need a solution to protect user information and handle normal network problems without corrupting game data. Using a centralized server with a web-based client supports these requirements while allowing The Gaming Room to reach users on desktop and mobile devices.

Design Constraints

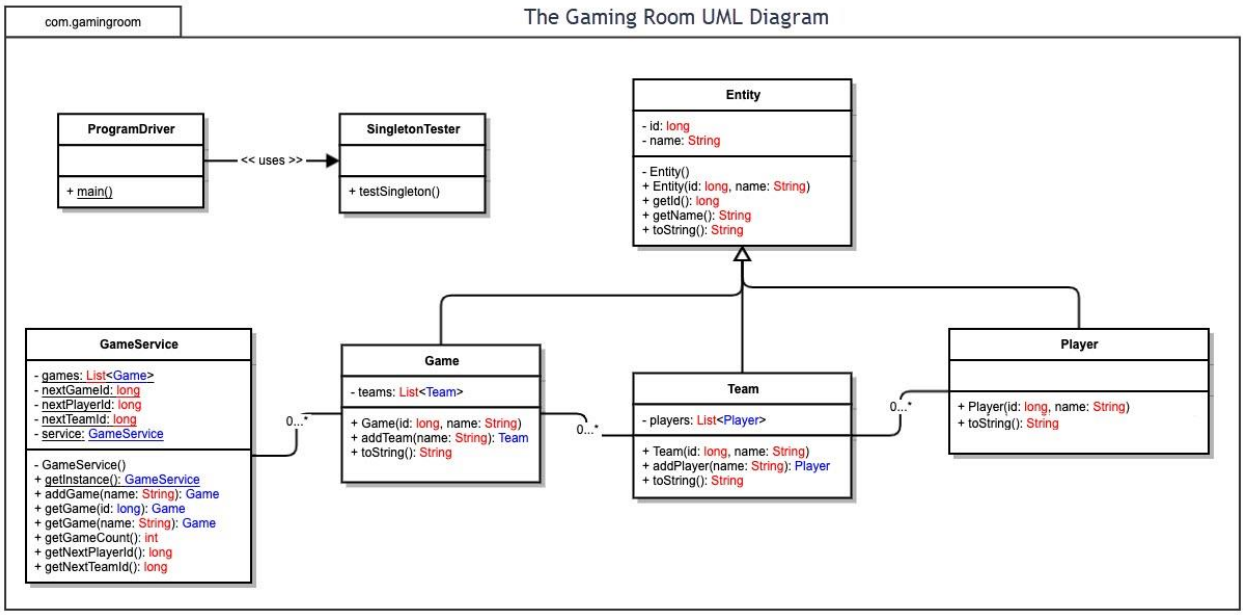
The design constraint is ensuring that the game operates seamlessly across multiple operating systems without necessitating a separate native application for each platform. Employing a responsive HTML, CSS, and JavaScript client minimizes duplication; however, comprehensive testing across various browsers, operating systems, screen sizes, and mobile input methods remains essential. The server architecture must support concurrent users and facilitate horizontal scaling to accommodate increased demand. Robust handling of network interruptions is required, as the game relies on continuous communication between clients and the server. Security considerations include consistent implementation of authentication, authorization, encrypted communications, input validation, secure sessions, and protection of stored user information across all platforms. Additionally, licensing costs, developer expertise, deployment complexity, and ongoing maintenance significantly influence the solution's total cost.

System Architecture View

I do not need a separate architecture diagram in this section because the template states that nothing additional is required here. I address the architecture through my distributed client-server recommendation and the UML domain model.

Domain Model

The domain model specifies the primary entities within the game and their interrelationships. The Entity class supplies shared information and behavior for all game objects. The Game class represents a single game instance, encompassing both teams and players. The Team class models a group competing within a game, whereas the Player class represents an individual participant. The GameService class oversees the collection of games and enforces the uniqueness of each game name. This design demonstrates inheritance through the Entity superclass, encapsulation via class fields and methods, and abstraction by centralizing common entity behavior in the superclass. These object-oriented principles reduce code duplication and improve maintainability as the application adapts to additional platforms.



Evaluation

Using your experience to evaluate the characteristics, advantages, and weaknesses of each operating platform (Linux, Mac, and Windows) as well as mobile devices, consider the requirements outlined below and articulate your findings for each. As you complete the table, keep in mind your client's requirements and look at the situation holistically, as it all must work together.

In each cell, remove the bracketed prompt and write your own paragraph response covering the indicated information.

Development Requirements	Mac	Linux	Windows	Mobile Devices

Server Side	macOS supports web hosting through standard Unix-based server software; however, Apple no longer offers a dedicated macOS Server product. As a result, deploying a production web server on macOS requires additional configuration and administration. The primary limitations are cost and server-management practicality, as macOS necessitates Apple hardware and is not typically selected for large-scale server farms. While suitable for development or small-scale deployments, macOS is less practical for scaling to thousands of users compared to server-oriented Linux distributions. Although Apple does not impose	Linux is well suited for web hosting due to its server-oriented architecture, compatibility with widely used web servers and cloud platforms, and flexibility in deployment on both physical servers and virtual machines. Distributions such as Ubuntu Server are available without traditional operating system license fees, whereas enterprise options like Red Hat Enterprise Linux require paid subscriptions for support and enterprise-level services. Additionally, Linux offers robust containerization and virtualization capabilities, which facilitate scalable game service	Windows Server provides a robust platform with comprehensive enterprise management, security, virtualization, and cloud integration features. It supports web application hosting and demonstrates effective scalability through virtualization and cloud infrastructure. A notable limitation is the associated licensing cost. Windows Server 2025 utilizes a core-based licensing model and may require additional Client Access Licenses, depending on the deployment scenario. Consequently, the total licensing cost for Windows Server is generally higher than that of a free Linux distribution,	. Mobile devices are suitable for use as clients; however, they are not practical as primary server platforms for games designed to support thousands of players. Android and iOS devices have limited battery life, storage capacity, connectivity, and resources compared to dedicated servers. Therefore, mobile devices should access the centralized web application via a responsive browser. This approach keeps game logic and persistent data on the server, allowing mobile users to participate without installing a platform-specific server.

	a separate per-core server license fee, the mandatory use of Apple hardware significantly increases infrastructure costs.	deployment. However, effective system administration often demands advanced Linux expertise, particularly in areas such as security, networking, monitoring, and production deployment.	particularly as infrastructure demands grow.	

Client Side	For macOS clients, applications should employ responsive HTML, CSS, and JavaScript, and must be tested in major browsers including Safari, Chrome, and Firefox. Development time increases due to the need for browser compatibility testing and validation of input devices, display scaling, and accessibility features. Web development expertise is required, eliminating the necessity for a separate native macOS application team. This strategy reduces costs by enabling a single web client to serve both macOS and other desktop platforms.	Linux clients require access to the same responsive web application; however, testing must encompass common Linux browser configurations and various desktop environments. The primary cost involves testing time and ensuring compatibility with current browser standards, rather than developing a separate Linux-native application. Developers must possess expertise in responsive web design, JavaScript, browser APIs, networking, and cross-browser testing.	Windows users can access the responsive web application through browsers such as Edge, Chrome, and Firefox. The development team is advised to test various screen resolutions, scaling settings, input devices, and browser versions. A separate Windows-native client is unnecessary if the application adheres to web standards. This approach reduces development time compared to maintaining a distinct Windows desktop application.	Android and iOS platforms necessitate additional responsive design and device testing due to variations in screen sizes, touch interfaces, performance characteristics, permissions, and network conditions across mobile browsers. Applications should implement mobile-first responsive layouts, touch-optimized controls, efficient asset management, and progressive loading strategies. Adopting a browser-based approach eliminates the need to develop two entirely separate native applications, thereby reducing development time and costs; however, comprehensive device and browser testing remains essential.

Development Tools	The primary tools for development are cross-platform web technologies, including HTML, CSS, JavaScript or TypeScript, a server-side language such as Java, C#, or JavaScript/Node.js, Git for source control, and a database system for persistent data. Visual Studio Code is compatible with macOS, and integrated development environments such as IntelliJ IDEA support Java development. Apple's Xcode is beneficial for native Apple development or device testing, but it is not required for browser-only clients.	Linux provides robust support for cross-platform web stacks and demonstrates particular strength in server-side development. Essential tools for this environment include Visual Studio Code, IntelliJ IDEA, Git, Java or Node.js runtimes, web servers or reverse proxies, containers, and automated testing utilities. The majority of these tools are freely available or distributed under open-source licenses. Proficiency in Linux-based operations is critical for effective deployment, monitoring, security, and automation.	Windows provides compatibility with development environments such as Visual Studio, Visual Studio Code, IntelliJ IDEA, Git, Java, .NET, Node.js, and browser developer tools. Visual Studio Community is offered at no cost for eligible individual and organizational use, as specified in its license terms. As a result, teams using Windows can develop both server and client components without incurring significant tool expenses, although professional or enterprise IDE editions may require paid subscriptions.	For mobile clients, the primary development stack should continue to utilize HTML, CSS, and JavaScript or TypeScript, as the application is delivered via the browser. Android Studio is recommended for Android-specific testing, while Xcode is appropriate for iOS-specific testing and for native development if required. The development team should employ both real devices and emulators or simulators to evaluate touch input, responsive layouts, performance, network variability, and accessibility. This approach necessitates mobile testing expertise but does not require dedicated native development teams unless The Gaming Room opts for native

				applications in the future.

Recommendations

Analyze the characteristics of and techniques specific to various systems architectures and make a recommendation to The Gaming Room. Specifically, address the following:

1. **Operating Platform**:
   Linux is recommended as the primary server operating platform for Draw It or Lose It. A Linux server efficiently hosts the web application, supports cloud and virtualization environments, and enables scalability without the operating-system licensing constraints associated with Windows Server. Implementing a browser-based client for Windows, macOS, Linux, Android, and iOS ensures that the application is accessible across all supported platforms.
2. Operating Systems Architectures:

A distributed client-server architecture is recommended. Clients interact with the application server via modern web browsers using HTTPS. The server architecture separates presentation and API services from business logic and persistent storage. Deployment can occur in virtual machines or containers behind a reverse proxy or load balancer, enabling scalability as the player base increases.

3. **Storage Management**:

A relational database such as PostgreSQL is recommended for storing persistent game, team, player, account, and configuration data. Frequently accessed temporary state should be maintained in an appropriate cache, while durable information remains in the database. The production design should incorporate database backups, replication, access controls, and encryption.

4. **Memory Management**:

Linux offers virtual memory, process isolation, and memory allocation, enabling multiple application services to operate efficiently. Applications should minimize unnecessary in-memory state to ensure that individual server instances can be added or removed without loss of critical game data. Implementing resource monitoring, setting memory limits for containers, and conducting load testing can identify memory leaks or excessive usage prior to production deployment.

5. **Distributed Systems and Networks**:

   Clients interact with the server using standard web protocols over the internet. HTTPS secures data transmission, and APIs enable browser clients to access game services. Load balancing and multiple application instances distribute incoming traffic. The application must address temporary outages and lost connections by detecting disconnections, retrying relevant requests, and preserving game state on the server. Monitoring and logging help identify failures and performance issues.

6. **Security**: Implement consistent security measures across all supported platforms. Servers must require robust authentication and enforce authorization for protected

operations. Passwords must be securely hashed instead of stored in plain text. Communications should utilize HTTPS or TLS, and sensitive data at rest must be encrypted when necessary. Employ input validation, parameterized database queries, secure session management, least-privilege permissions, regular dependency updates, comprehensive logging, monitoring, and routine vulnerability assessments to mitigate security risks. Server-side authorization is critical, as client-side controls are susceptible to circumvention.